

Development of a Prompt-Based Application Using ChatGPT for Daily Task Organization

Abstract

This report presents the design and development of a prompt-based application using ChatGPT to organize daily tasks effectively. The application demonstrates how prompt engineering can be used to automate task management, improve productivity, and support practical problem-solving. The report explains the evolution of prompts from simple instructions to advanced contextual prompts and analyzes their outputs. The study highlights the role of large language models (LLMs) in creating intelligent productivity systems that assist users in planning schedules, prioritizing tasks, generating reminders, and improving workflow efficiency.

1. Introduction

Prompt-based applications use carefully designed textual instructions to interact with large language models such as ChatGPT. These applications allow users to automate workflows, generate personalized recommendations, and solve practical problems efficiently.

Task organization is one of the most useful real-world applications of prompt engineering. By integrating structured prompts with AI systems, users can simplify daily planning, prioritize activities, and improve productivity.

This project demonstrates how ChatGPT can be used to develop a prompt-based task management application that evolves from basic prompting methods to more advanced prompt engineering techniques.

2. Objectives

The objectives of this project are:

- To develop a prompt-based application using ChatGPT.
- To organize daily tasks using AI-generated responses.
- To demonstrate the progression from simple prompts to advanced prompts.
- To compare outputs generated by different prompt styles.
- To enhance creativity and practical problem-solving skills.
- To explore the capabilities of large language models in productivity applications.

3. Background of Prompt Engineering

Prompt engineering refers to the process of designing and optimizing prompts to guide AI systems toward producing accurate and useful outputs.

Effective prompt engineering improves:

- Response quality
- Task relevance
- Context understanding
- Output consistency
- Creativity and reasoning

Large language models interpret prompts differently depending on wording, structure, context, and constraints.

4. Application Overview

The proposed application is a smart daily task organizer powered by ChatGPT. The application helps users:

- Create daily schedules
- Prioritize tasks
- Set reminders
- Manage deadlines
- Generate productivity suggestions
- Optimize workflow planning

The application accepts user inputs and generates personalized task plans using AI prompts.

5. System Architecture

The application architecture consists of:

1. User Input Interface
2. Prompt Processing Layer
3. ChatGPT API Integration
4. Response Generation Engine
5. Task Management Module
6. Output Display System

The prompt processing layer is responsible for converting user requests into structured prompts.

6. Development of Prompt Designs

The application demonstrates prompt evolution across multiple stages:

1. Basic Prompts
2. Structured Prompts
3. Contextual Prompts
4. Role-Based Prompts
5. Multi-Step Prompts
6. Personalized Prompts

Each stage improves response quality and user experience.

7. Simple Prompt Example

Example Prompt:

"Create a to-do list for today."

Typical Output:

- Check emails
- Attend meetings
- Complete assignments
- Exercise
- Read for 30 minutes

Analysis:

The response is generic because the prompt lacks context and personalization.

8. Structured Prompt Example

Example Prompt:

"Create a daily schedule for a college student with classes from 9 AM to 2 PM, including study time, exercise, and

relaxation."

Typical Output:

- 7:00 AM – Wake up
- 8:00 AM – Breakfast
- 9:00 AM – Classes
- 3:00 PM – Study Session
- 6:00 PM – Exercise
- 8:00 PM – Relaxation

Analysis:

The structured prompt generates more organized and relevant outputs.

9. Contextual Prompt Example

Example Prompt:

"I have exams next week and need help organizing my daily tasks to maximize study efficiency while maintaining healthy sleep habits."

Typical Output:

- Prioritized study schedule
- Break intervals
- Sleep recommendations
- Revision reminders
- Time allocation suggestions

Analysis:

Contextual prompts significantly improve personalization and usefulness.

10. Role-Based Prompt Example

Example Prompt:

"Act as a productivity coach and create a balanced daily routine for a software engineering student preparing for internships."

Typical Output:

- Coding practice schedule
- Resume preparation
- Interview preparation
- Project development tasks
- Health and wellness recommendations

Analysis:

Role-based prompts improve professionalism and domain-specific guidance.

11. Multi-Step Prompt Example

Example Prompt:

1. Analyze my current schedule.
2. Identify time-wasting activities.
3. Suggest improvements.
4. Create an optimized daily plan.

Typical Output:

- Productivity analysis
- Efficiency recommendations
- Time-blocked schedule
- Improvement strategies

Analysis:

Multi-step prompting improves reasoning depth and planning quality.

12. Comparative Prompt Analysis

Different prompting styles produce different levels of output quality.

Broad prompts often generate generic responses, while refined prompts improve:

- Specificity
- Relevance
- Accuracy
- Productivity value
- User satisfaction

Prompt engineering directly impacts application performance.

13. Technologies Used

Technologies used in the application include:

- Python
- OpenAI ChatGPT API
- Flask / Streamlit
- HTML/CSS
- JSON
- REST APIs

These technologies enable frontend interaction and backend AI integration.

14. Sample Python Code

The following sample demonstrates ChatGPT integration for task management.

```
import openai

openai.api_key = "YOUR_API_KEY"

prompt = """
Create a daily task schedule for a university student
with classes from 9 AM to 2 PM. Include study time,
exercise, and relaxation.
"""

response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[
        {"role": "user", "content": prompt}
    ]
)

print(response["choices"][0]["message"]["content"])
```

15. Benefits of the Application

- Advantages include:
- Improved productivity
 - Better task prioritization
 - Reduced stress
 - Personalized planning
 - Enhanced time management
 - Increased workflow efficiency

16. Limitations

- Limitations include:
- Dependency on internet connectivity
 - API usage costs
 - Potential AI inaccuracies
 - Limited understanding of emotional context
 - Privacy concerns with sensitive data

17. Future Scope

- Future improvements may include:
- Voice-based task interaction
 - Calendar synchronization
 - AI-generated reminders
 - Mobile application support
 - Emotion-aware scheduling
 - Team collaboration features

18. Conclusion

This project demonstrates how prompt engineering and ChatGPT can be used to develop intelligent productivity applications. By evolving prompts from simple instructions to advanced contextual prompts, the application produces more personalized, relevant, and effective outputs.

The report highlights the importance of prompt design in improving AI interactions and demonstrates the practical value of large language models in solving everyday productivity challenges.

19. Comparative Evaluation Table

Prompt Type	Output Quality	Personalization	Complexity
Basic Prompt	Moderate	Low	Low
Structured Prompt	High	Medium	Medium
Contextual Prompt	Very High	High	High
Role-Based Prompt	Very High	High	High
Multi-Step Prompt	Excellent	Very High	Advanced

20. References

- OpenAI API Documentation

- [Prompt Engineering Guide](#)
- [Research Papers on Large Language Models](#)
- [Python Official Documentation](#)
- [Flask Web Framework Documentation](#)
- [Human-AI Interaction Studies](#)